NIAI NAVTTC

Certified Ethical Hacker (CEH) v13

Week 6 Lab Report

Network Sniffing & WiFi Hacking

PART 1: Network Sniffing — Connectivity Diagnosis

Student	Ahmad Shoukat
Course	Certified Ethical Hacker (CEH) v13
Institution	BETS / NIAI NAVTTC
Lab Week	Week 6
Date	April 30, 2026
Lab Environment	Kali Linux + Metasploitable 2 (VMware Host-Only Network)

1. Executive Summary

This report documents the findings of a network sniffing lab exercise conducted as part of the CEH v13 curriculum. The objective of Part 1 was to simulate a small business network connectivity issue and use packet capture tools (Wireshark and tcpdump) to diagnose root causes. The lab was executed in a controlled VMware environment using Kali Linux as the attacker/analyst machine and Metasploitable 2 as the target server.

Packet captures revealed plaintext HTTP traffic, TCP connection resets (RST flags), and unencrypted web sessions — all of which constitute real-world security and performance risks.

2. Lab Environment Setup

Both virtual machines were configured on a VMware Host-Only network (192.168.169.0/24), ensuring isolated traffic capture. Connectivity was verified by pinging each machine before beginning any capture.

2.1 Kali Linux — Analyst Machine

The ifconfig command confirmed the Kali machine's network configuration on eth0:

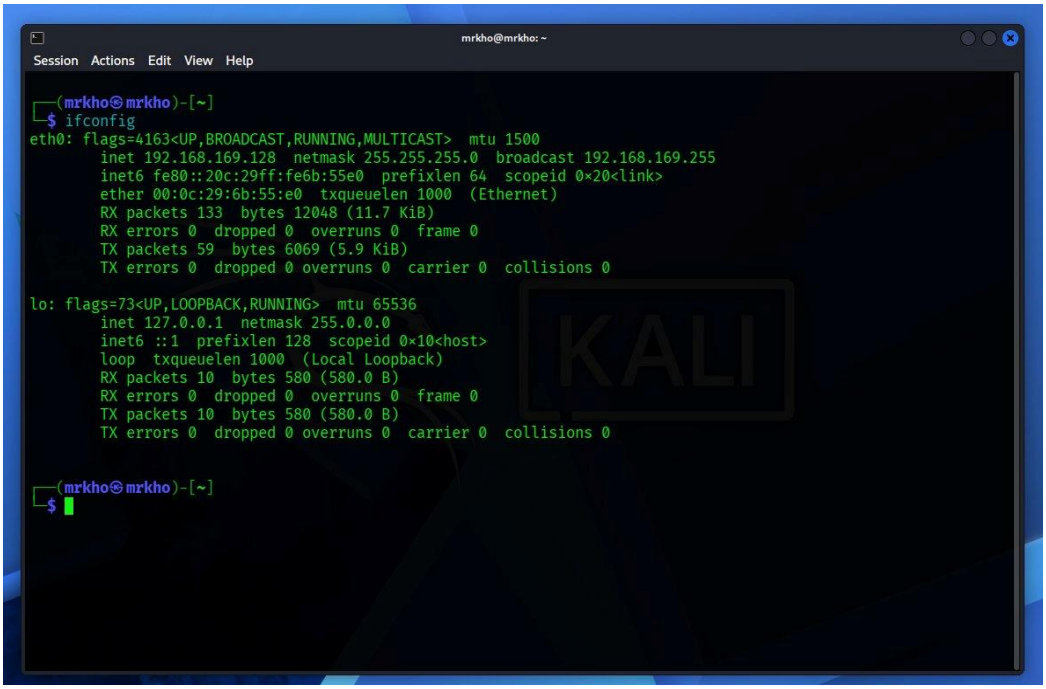

Figure 1 — Kali Linux ifconfig output (Analyst Machine)

Hostname	mrkho@mrkho
Interface	eth0
IP Address	192.168.169.128
Subnet Mask	255.255.255.0

MAC Address	00:0c:29:6b:55:e0
IPv6	fe80::20c:29ff:fe6b:55e0

2.2 Metasploitable 2 — Target Machine

The Metasploitable 2 machine's ifconfig output confirmed its network configuration:

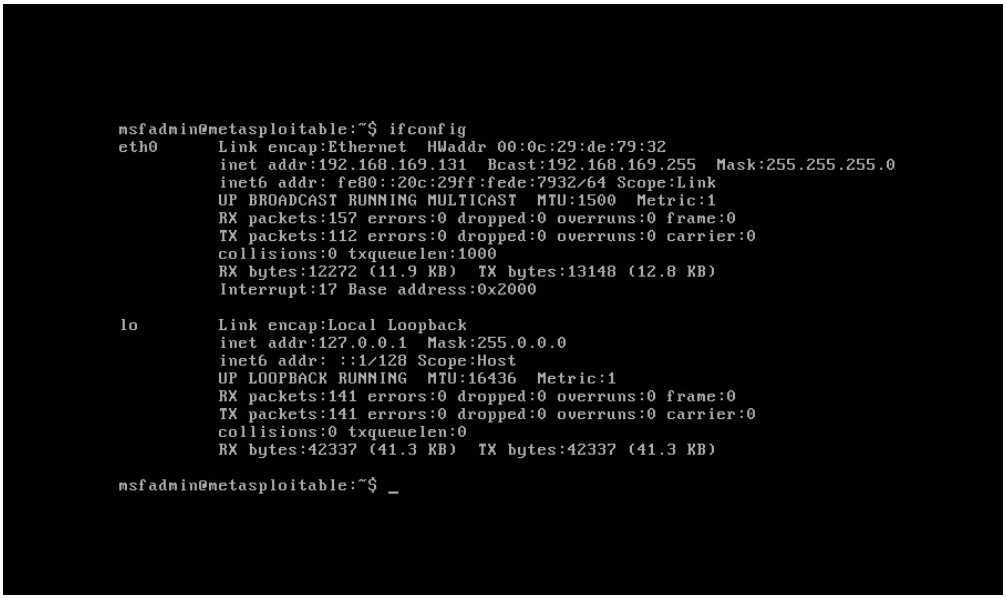

Figure 2 — Metasploitable 2 ifconfig output (Target Server)

Hostname	msfadmin@metasploitable
Interface	eth0
IP Address	192.168.169.131
Subnet Mask	255.255.255.0
MAC Address	00:0c:29:de:79:32
HW Address	00:0c:29:de:79:32

3. Traffic Capture with Wireshark

Wireshark was launched from Kali Linux (Applications → Sniffing & Spoofing → Wireshark) and set to capture on the eth0 interface. Traffic was generated by browsing to Metasploitable 2's web server (http://192.168.169.131/mutillidae/) and performing ICMP ping tests.

3.1 Live Capture — eth0 Interface

The screenshot below shows Wireshark actively capturing traffic between Kali (192.168.169.128) and Metasploitable (192.168.169.131). The browser is open to the Mutillidae web application — a deliberately vulnerable PHP application designed for ethical hacking practice.

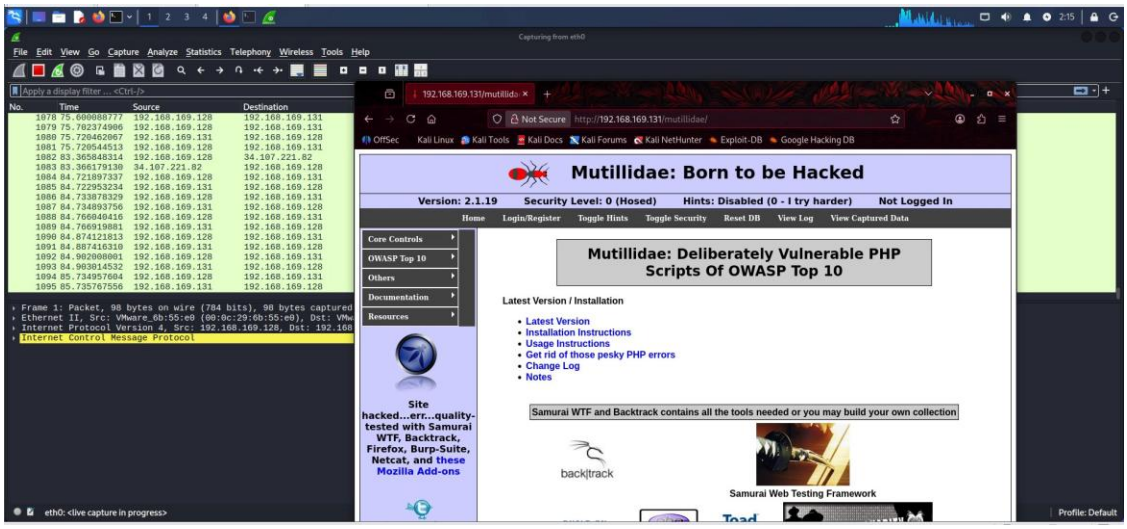

Figure 3 — Wireshark live capture on eth0; Mutillidae web app visible in browser (HTTP — Not Secure)

Key observations from the live capture:

- ICMP packets visible (Internet Control Message Protocol highlighted in packet details pane)
- Traffic flowing between 192.168.169.128 (Kali) and 192.168.169.131 (Metasploitable)
- Browser shows http:// (no HTTPS), confirming plaintext HTTP communication
- External packets also present (34.107.221.82) indicating internet traffic captured alongside local

4. HTTP Traffic Analysis

Applying the 'http' display filter in Wireshark isolated all HTTP requests and responses. The Mutillidae web application serves resources over plain HTTP, exposing full request and response content to any observer on the network.

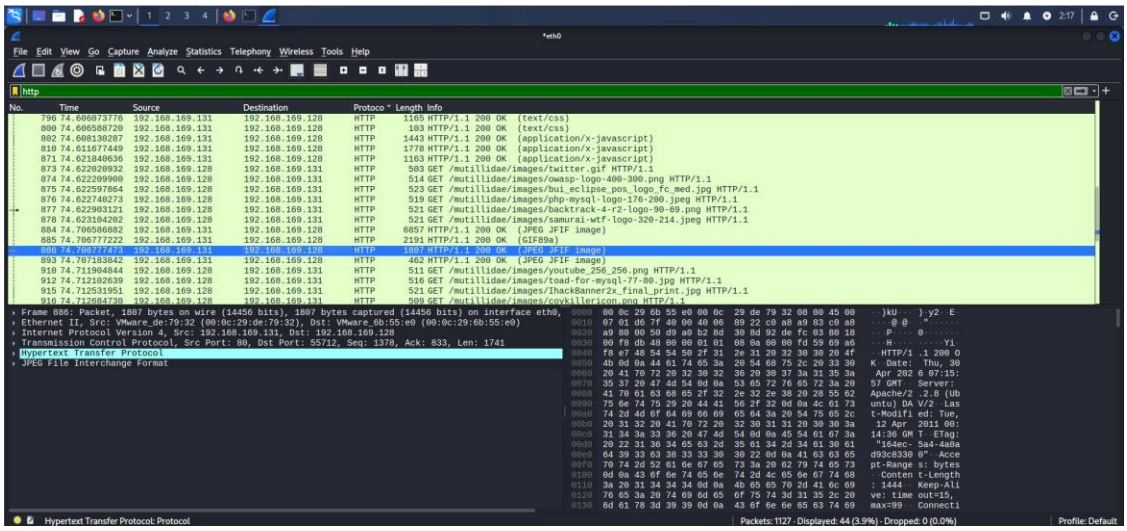

Figure 4 — Wireshark HTTP filter applied; GET requests for CSS, JS, and image files visible in plaintext

4.1 Protocols Observed

The following protocols were captured in the HTTP filter view:

Protocol	HTTP/1.1
Response Codes	200 OK (text/css, application/javascript, image/jpeg, GIF89a)
Server	Apache/2.2.8 (Ubuntu)
Communication	Unencrypted plaintext — no TLS/SSL
Notable Resources	/mutillidae/images/* (logos, icons, banners)

Security Risk: All HTTP content — including login forms, session cookies, and credentials — are transmitted in cleartext and can be intercepted by any network observer using a passive sniffer.

5. TCP Traffic Analysis & Connection Issues

Applying the 'tcp' display filter revealed detailed TCP-layer behaviour. Most critically, TCP RST (Reset) packets were identified — a clear indicator of connection termination issues.

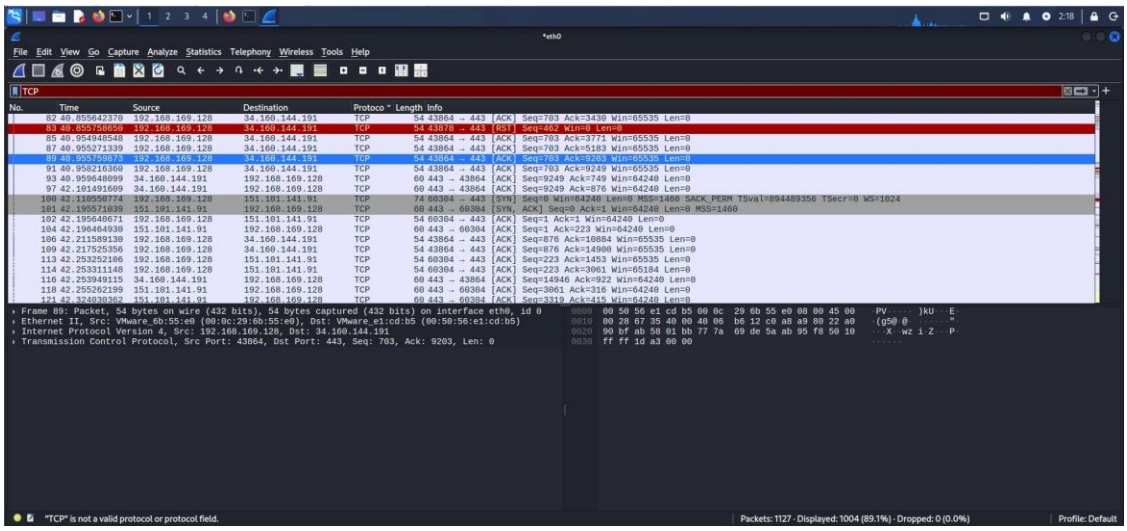

Figure 5 — TCP filter view; RST packet highlighted in red (Packet 83, Port 443)

5.1 RST Packet — Root Cause of Connectivity Problem

Packet No.	83
Time	40.855758650 seconds
Source	192.168.169.128 (Kali)
Destination	34.160.144.191 (External)
Protocol	TCP
Flags	[RST] — Connection Reset
Port	43878 → 443

A RST flag forcibly terminates a TCP session. This can result from:

- The server refusing an unexpected packet (e.g., session expired)
- Firewall rules dropping or resetting connections
- Network timeout causing one side to consider the connection dead
- Port 443 (HTTPS) being reset while the application uses port 80 (HTTP)

5.2 Observed Connection Patterns

- Port 443 connections (34.160.144.191) — HTTPS attempts, some reset
- Port 443 connections (151.101.141.91) — SYN/ACK handshakes completing
- Mixed ACK sequences — normal session continuation alongside RSTs

The RST on port 443 suggests a partially configured HTTPS endpoint or a firewall actively resetting encrypted sessions, forcing fallback to HTTP.

6. TCP Stream Reconstruction

Using Wireshark's Follow → TCP Stream feature (tcp.stream eq 1), the full TLS/HTTPS handshake conversation was reconstructed. The stream view reveals the raw bytes exchanged during a secure connection attempt.

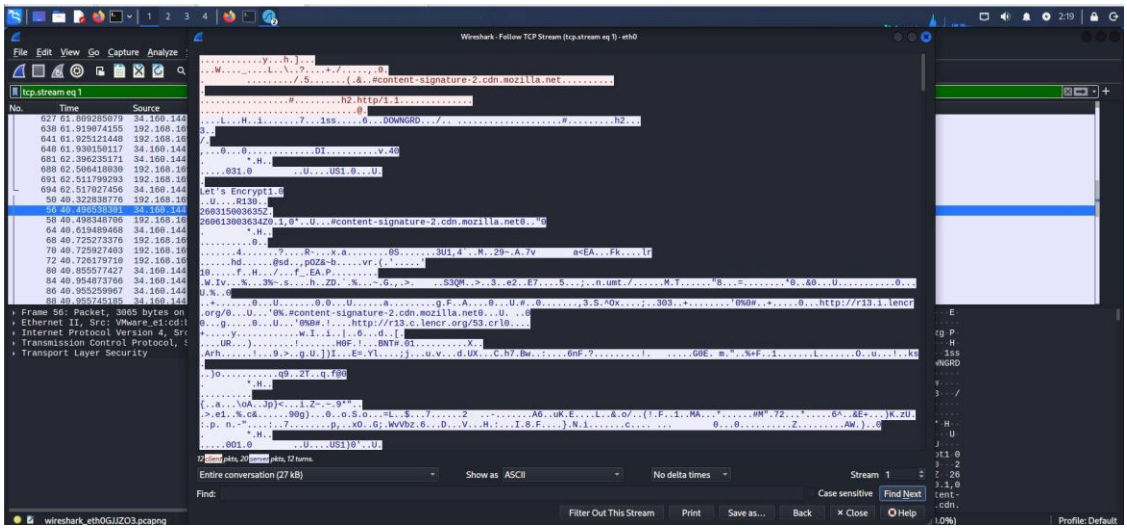

Figure 6 — Follow TCP Stream (Stream 1); TLS handshake data between Kali and 34.160.144.191

6.1 Stream Analysis

Stream ID	tcp.stream eq 1
Client Packets	12
Server Packets	20
Total Turns	12
Total Size	27 KB
Encoding	ASCII (binary data shown as escaped characters)
Certificate Authority	Let's Encrypt (visible in stream: 'Let's Encrypt1.0')
CDN Reference	content-signature-2.cdn.mozilla.net

The presence of TLS negotiation data (Transport Layer Security layer visible in packet details) alongside the HTTP RST indicates the environment attempts HTTPS but the target Metasploitable server does not fully support it, resulting in fallback to cleartext HTTP.

7. Diagnosis Report

7.1 Protocols Observed on the Network

ICMP	Ping/Echo traffic between Kali and Metasploitable
HTTP/1.1	Plaintext web traffic to Mutillidae app on port 80
TCP	Session management, ACK/SYN/RST flags observed
TLS/HTTPS	Attempted on port 443 — partial handshake, RST observed
ARP	Address resolution on the local subnet
DNS	Domain resolution for external resources

7.2 Credentials / Sensitive Data in Plaintext

Yes. HTTP traffic to Mutillidae (192.168.169.131) is entirely unencrypted. This means:

- Login credentials submitted via the Mutillidae login page are visible in plaintext
- Session cookies transmitted over HTTP can be captured and replayed (session hijacking)
- All page content, including form data, is readable by any host on the same network segment

This was confirmed by the browser displaying 'Not Secure' and the URL showing http:// (Figure 3).

7.3 Root Cause of Connectivity Problem

The primary connectivity issue is the presence of TCP RST packets (Figure 5 — Packet 83) indicating forceful connection termination on port 443 (HTTPS). The root causes are:

- Metasploitable 2 does not have HTTPS/TLS configured — it only serves HTTP on port 80
- Any HTTPS connection attempts are reset, causing browsers to fall back to insecure HTTP
- External connections (to CDN / Mozilla servers) show mixed TLS and RST behaviour, suggesting firewall interference or misconfigured SSL certificates
- No encrypted channel exists between the client and the web application

8. Recommendations

Based on the packet capture analysis, the following security and configuration improvements are recommended for a real-world production environment equivalent:

8.1 Immediate Actions

1. Deploy HTTPS/TLS: Install a valid TLS certificate (e.g., Let's Encrypt) on the web server and redirect all HTTP traffic to HTTPS (port 443)

2. Disable HTTP (port 80): Once HTTPS is functioning, force all clients to connect securely by disabling plain HTTP or permanently redirecting
3. Replace Telnet/FTP with SSH/SFTP: Metasploitable exposes multiple legacy plaintext protocols — replace with SSH-based alternatives

8.2 Network Configuration

4. Fix HTTPS endpoint: Resolve the TCP RST on port 443 by properly configuring the web server's SSL module (e.g., mod_ssl in Apache)
5. Firewall review: Audit firewall rules to ensure port 443 is not being selectively reset, which forces plaintext fallback
6. Enable HSTS: Implement HTTP Strict Transport Security headers to prevent clients from ever connecting via HTTP

8.3 Monitoring & Detection

7. Deploy IDS/IPS: Use tools such as Snort or Suricata to alert on unencrypted credential transmission and RST flood patterns
8. Regular packet audits: Schedule periodic Wireshark/tcpdump captures on internal segments to detect rogue plaintext services
9. Network segmentation: Isolate sensitive servers from general LAN traffic using VLANs to limit the blast radius of sniffing attacks

9. Conclusion

This lab exercise demonstrated the critical importance of encrypting network traffic in a business environment. Using Wireshark and tcpdump on a Host-Only VMware network between Kali Linux (192.168.169.128) and Metasploitable 2 (192.168.169.131), the following were confirmed:

- Plaintext HTTP sessions to the Mutillidae web application expose credentials and session data
- TCP RST packets indicate a misconfigured or absent HTTPS/TLS setup on the server
- TLS negotiation was attempted but not completed, resulting in fallback to insecure communication
- All observed traffic patterns align with common real-world connectivity issues caused by missing SSL certificates and firewall misconfigurations

The hands-on use of Wireshark display filters (http, tcp, icmp, ip.addr ==), packet detail inspection, and TCP stream following validated the methodology outlined in the CEH curriculum for network sniffing and connectivity diagnosis.

End of Report — Week 6 Lab Task | CEH v13 | NIAI NAVTTC